

API Design Document

Project: AI-Powered Intelligent Interview Question Management System

Version: 1.0

Prepared By: Utkarsh Krishna Tripathi

Date: 02/07/2026

Table of Contents

- 1. Introduction
- 2. API Standards
- 3. Common Request/Response Format
- 4. API Response Status Codes
- 5. API Endpoint Summary
- 6. Authentication APIs
- 7. User Management APIs
- 8. Role & Permission APIs
- 9. Question Management APIs
- 10. Metadata APIs
- 11. AI Search APIs
- 12. Search History APIs
- 13. Audit Log APIs

1. Introduction

1.1 Purpose

This document defines the REST API design for the AI-Powered Intelligent Interview Question Management System. It provides the specifications for API endpoints, request and response formats, authentication, authorization, and error handling required for communication between the frontend, backend, and AI services.

1.2 Scope

This document covers the design of all backend APIs used for user authentication, user and role management, interview question management, metadata management, AI-powered semantic search, search history, and audit logging. It also defines API standards, security mechanisms, and response formats.

2. API Standards

2.1 Naming Conventions

The APIs follow RESTful design principles with the following standards:

- RESTful endpoints
 - JSON request and response format
 - HTTPS communication
 - API versioning (/api/v1)
-

2.2 HTTP Methods

HTTP Methods

Method	Description
GET	Retrieve resources
POST	Create a new resource
PUT	Update an existing resource
PATCH	Partially update an existing resource
DELETE	Remove a resource

3. Common Request/Response Format

Success Response

```
{  
  "success": true,  
  "message": "Request processed successfully.",  
  "data": {  
    // Response payload  
  }  
}
```

Error Response

```
{  
  "success": false,  
  "message": "Validation failed.",  
  "errors": [  
    "Title is required.",  
    "Description cannot be empty."  
  ]  
}
```

4. API Response Status Codes

Status Code	Meaning
200	Request completed successfully.
201	Resource created successfully.
204	Request completed successfully with no response body.
400	Invalid request or validation failed.
401	Authentication is required or failed.
403	Authenticated user does not have permission.
404	Requested resource does not exist.
409	Request conflicts with existing data.
500	Unexpected server-side error.

5. API Endpoint Summary

Module	Base Endpoint	APIs	Description
Authentication	/api/v1/auth	2	User authentication and profile management
User Management	/api/v1/users	6	Manage user accounts and status
Role & Permission	/api/v1/role-types, /api/v1/role-permissions	3	Manage roles and permissions
Question Management	/api/v1/questions	5	Manage interview questions
Metadata	/api/v1/metadata	5	Manage metadata values
AI Search	/api/v1/search	2	AI-powered semantic search using RAG
Search History	/api/v1/search-history	2	View previous search records
Audit Log	/api/v1/audit-logs	1	View system audit logs

6. Authentication APIs

6.1 Google Login

- **Endpoint:** POST /api/v1/auth/google-login
 - **Description:** Authenticates a user using Google OAuth and returns an access token upon successful authentication.
 - **Authorization:** Public
 - **Request Body:** Google OAuth ID Token.
 - **Success Response:** Returns the authenticated user's information along with the access token.
 - **Error Responses:** 400, 401, 403, 500
-

6.2 Get User Profile

- **Endpoint:** GET /api/v1/auth/profile
 - **Description:** Retrieves the profile information of the currently authenticated user.
 - **Authorization:** Root user, Admin, Question Manager, Viewer
 - **Request Body:** None
 - **Success Response:** Returns the authenticated user's profile details.
 - **Error Responses:** 401, 403, 404, 500
-

Business Rules

- Authentication is performed using Google OAuth 2.0.
- Only users with a valid **@getcarnera.com** email address can access the system.
- Only active user accounts are allowed to log in.

- User roles are determined from the `ROLE_TYPE` table after successful authentication.
 - A JWT access token is generated for all authenticated users.
-

Validation Rules

- A valid Google OAuth ID Token is required.
- The authenticated email address must belong to the **@getcarnera.com** domain.
- The user account must exist and be active.
- A valid JWT access token is required to access protected APIs.

7. User Management APIs

7.1 Get All Users

Endpoint: GET /api/v1/users

Description: Retrieves a list of all registered users.

Authorization: Root user, Admin

Request Body: None

Success Response: Returns the list of registered users.

Error Responses: 401, 403, 500

7.2 Get User by ID

Endpoint: GET /api/v1/users/{id}

Description: Retrieves detailed information about a specific user.

Authorization: Root user, Admin

Path Parameter: userId

Request Body: None

Success Response: Returns the requested user's details.

Error Responses: 400, 401, 404, 500

7.3 Create User

Endpoint: POST /api/v1/users

Description: Creates a new user account and assigns a role.

Authorization: Root user, Admin

Request Body: User details including name, email, role, and account status.

Success Response: User created successfully.

Error Responses: 400, 401, 403, 409, 500

7.4 Update User

Endpoint: PUT /api/v1/users/{id}

Description: Updates the information of an existing user.

Authorization: Root user, Admin

Path Parameter: userId

Request Body: Updated user information.

Success Response: User updated successfully.

Error Responses: 400, 401, 404, 500

7.5 Update User Status

Endpoint: PATCH /api/v1/users/{id}/status

Description: Activates or deactivates a user account.

Authorization: Root user, Admin

Path Parameter: userId

Request Body: Updated account status.

Success Response: User status updated successfully.

Error Responses: 400, 401, 404, 500

7.6 Delete User

Endpoint: DELETE /api/v1/users/{id}

Description: Deletes an existing user account from the system.

Authorization: Root user, Admin

Path Parameter: userId

Request Body: None

Success Response: User deleted successfully.

Error Responses: 400, 401, 404, 500

Business Rules

- Only Root user and Admin users can perform user management operations.
 - Every user must have a unique email address.
 - Each user must be assigned exactly one role.
 - Only active users are allowed to access the system.
 - All user management activities are recorded in the Audit Log.
-

Validation Rules

- userId must be a valid GUID.
- Name is mandatory.
- Email must be a valid @getcarnera.com email address.
- A valid role must be assigned to the user.
- The account status (IsActive) must be a valid Boolean value.

8. Role & Permission APIs

8.1 Get All Role Types

Endpoint: GET /api/v1/role-types

Description: Retrieves all available role types in the system.

Authorization: Root user, Admin

Request Body: None

Success Response: Returns the list of available role types.

Error Responses: 401, 403, 500

8.2 Get Role Permissions

Endpoint: GET /api/v1/role-permissions/{roleId}

Description: Retrieves all permissions assigned to a specific role.

Authorization: Root user, Admin

Path Parameter: roleId

Request Body: None

Success Response: Returns the permission matrix for the selected role.

Error Responses: 400, 401, 404, 500

8.3 Update Role Permissions

Endpoint: PUT /api/v1/role-permissions/{roleId}

Description: Updates the permissions assigned to a specific role.

Authorization: Root user, Admin

Path Parameter: roleId

Request Body: Updated permission details.

Success Response: Role permissions updated successfully.

Error Responses: 400, 401, 404, 500

Business Rules

- Only Root and admin users can view and modify role permissions.
 - Every role must have a unique set of permissions.
 - Permissions are assigned at the module level (View, Create, Update, Delete).
 - Permission changes take effect immediately after a successful update.
 - All permission changes are recorded in the Audit Log.
-

Validation Rules

- roleTypeId must be a valid GUID.
- The specified role must exist.
- Permission values (CanView, CanCreate, CanUpdate, CanDelete) must be Boolean values.
- A permission entry must reference a valid system module.

9. Question Management APIs

9.1 Get All Questions

Endpoint: GET /api/v1/questions

Description: Retrieves all interview questions.

Authorization: Root user, Admin, Question Manager, Viewer

Request Body: None

Success Response: Returns the list of interview questions.

Error Responses: 401, 403, 500

9.2 Get Question by ID

Endpoint: GET /api/v1/questions/{id}

Description: Retrieves the details of a specific interview question.

Authorization: Root user, Admin, Question Manager, Viewer

Path Parameter: questionId

Request Body: None

Success Response: Returns the requested question.

Error Responses: 400, 401, 404, 500

9.3 Create Question

Endpoint: POST /api/v1/questions

Description: Creates a new interview question with its answer and associated metadata.

Authorization: Root user, Admin, Question Manager

Request Body: Question details including title, description, answer, and metadata.

Success Response: Question created successfully.

Error Responses: 400, 401, 403, 500

9.4 Update Question

Endpoint: PUT /api/v1/questions/{id}

Description: Updates an existing interview question.

Authorization: Root user, Admin, Question Manager

Path Parameter: questionId

Request Body: Updated question information.

Success Response: Question updated successfully.

Error Responses: 400, 401, 404, 500

9.5 Delete Question

Endpoint: DELETE /api/v1/questions/{id}

Description: Performs a soft delete by marking the question as deleted.

Authorization: Root user, Admin, Question Manager

Path Parameter: questionId

Request Body: None

Success Response: Question deleted successfully.

Error Responses: 400, 401, 404, 500

Business Rules

- Only Root user, Admin and Question Manager can create, update, or delete questions.
- Viewer users have read-only access.
- Every question must be associated with a valid creator.
- Deleted questions are soft deleted using the IsDeleted flag.

- Question changes must be recorded in the Audit Log.
-

Validation Rules

- Question title is mandatory.
- Answer field cannot be empty.
- CreatedBy must reference an existing user.
- QuestionId must be a valid GUID.
- Metadata references must exist before being associated with a question.

10. Metadata APIs

10.1 Get All Metadata

Endpoint: GET /api/v1/metadata

Description: Retrieves all metadata values available in the system.

Authorization: Root user, Admin, Question Manager, Viewer

Request Body: None

Success Response: Returns the list of metadata values.

Error Responses: 401, 403, 500

10.2 Get Metadata by Type

Endpoint: GET /api/v1/metadata?type={type}

Description: Retrieves metadata values for a specific category such as Organization, Manager, Programming Language, Framework, Concept, Difficulty, or Library.

Authorization: Root user, Admin, Question Manager, Viewer

Query Parameter: type

Request Body: None

Success Response: Returns metadata values for the specified type.

Error Responses: 400, 401, 404, 500

10.3 Create Metadata

Endpoint: POST /api/v1/metadata

Description: Creates a new metadata value.

Authorization: Root user, Admin, Question Manager

Request Body: Metadata details.

Success Response: Metadata created successfully.

Error Responses: 400, 401, 403, 409, 500

10.4 Update Metadata

Endpoint: PUT /api/v1/metadata/{id}

Description: Updates an existing metadata value.

Authorization: Root user, Admin, Question Manager

Path Parameter: metadataId

Request Body: Updated metadata information.

Success Response: Metadata updated successfully.

Error Responses: 400, 401, 404, 500

10.5 Delete Metadata

Endpoint: DELETE /api/v1/metadata/{id}

Description: Deletes a metadata value from the system.

Authorization: Root user, Admin, Question Manager

Path Parameter: metadataId

Request Body: None

Success Response: Metadata deleted successfully.

Error Responses: 400, 401, 404, 500

Business Rules

- Metadata values are categorized by type (Organization, Manager, Programming Language, Framework, Concept, Difficulty, Library, etc.).
- Duplicate metadata values within the same type are not allowed.
- Only Root user, Admin and Question Manager can create or modify metadata.
- Metadata associated with existing questions cannot be deleted until the association is removed.

- All metadata changes must be recorded in the Audit Log.
-

Validation Rules

- Metadata type is mandatory.
- Metadata value is mandatory.
- MetadataId must be a valid GUID.
- Metadata type and value combination must be unique.

11. AI Search APIs

11.1 AI Semantic Search

Endpoint: POST /api/v1/search

Description: Performs AI-powered semantic search using the Retrieval-Augmented Generation (RAG) pipeline. The system converts the user's natural language query into vector embeddings using the Ollama embedding model (nomic-embed-text), performs vector similarity search in SQL Server, retrieves the most relevant interview questions, sends the retrieved context to the Groq LLM, and returns an AI-generated response along with the matching questions.

Authorization: Root user, Admin, Question Manager, Viewer

Request Body: Search query and optional filters.

Success Response: Returns relevant interview questions along with the AI-generated response from the Groq LLM.

Error Responses: 400, 401, 500

11.2 Get Related Questions

Endpoint: GET /api/v1/questions/related/{questionId}

Description: Retrieves interview questions that are semantically similar to the selected question using SQL Server Vector Search. The endpoint returns related questions based on vector similarity without generating a new AI response.

Authorization: Root user, Admin, Question Manager, Viewer

Path Parameter: questionId

Request Body: None

Success Response: Returns a list of related interview questions.

Error Responses: 400, 401, 404, 500

Business Rules

- Only authenticated users can perform AI-powered searches.

- AI responses are generated using the Retrieval-Augmented Generation (RAG) pipeline.
 - Groq LLM generates the final context-aware response using the retrieved content.
 - Search history is automatically stored after every successful search.
 - Metadata filters are applied before generating the AI response.
-

Validation Rules

- Search query cannot be empty.
- Search query must be within the supported character limit.
- Metadata filters must reference valid metadata values.
- questionId must be a valid GUID for related question retrieval.

12. Search History APIs

12.1 Get Search History

Endpoint: GET /api/v1/search-history

Description: Retrieves the search history of the authenticated user.

Authorization: Viewer

Request Body: None

Success Response: Returns the list of previous search records.

Error Responses: 401, 403, 500

12.2 Get Search History by ID

Endpoint: GET /api/v1/search-history/{id}

Description: Retrieves the details of a specific search history record, including the search query, applied filters, AI-generated response, and search timestamp.

Authorization: Viewer

Path Parameter: searchHistoryId

Request Body: None

Success Response: Returns the requested search history record.

Error Responses: 400, 401, 404, 500

Business Rules

- Search history is automatically created after every successful AI search.
- Users can view only their own search history.
- Search records include the search query, applied filters, AI response, and search timestamp.
- Search history is maintained for auditing and future reference.

Validation Rules

- searchHistoryId must be a valid GUID.
- The requested search history record must exist.
- A valid JWT access token is required.

13. Audit Log APIs

13.1 Get Audit Logs

Endpoint: GET /api/v1/audit-logs

Description: Retrieves audit log records based on optional search criteria and pagination.

Authorization: Root User, Admin

Query Parameters (Optional):

- severity
- eventType
- fromDate
- toDate

Request Body: None

Success Response: Returns a paginated list of audit log records.

Error Responses: 401, 403, 500

Business Rules

- Audit logs are generated automatically by the system.
- Audit logs are read-only and cannot be created, updated, or deleted through any API.
- Only Root User and Admin can access audit logs.
- Audit logs are used for monitoring, troubleshooting, and security analysis.
- The API supports filtering and pagination to efficiently retrieve audit records.

Validation Rules

- Query parameters must be valid, if provided.
- fromDate must not be later than toDate.
- severity must match a supported severity level.
- eventType must match a valid audit event type.
- A valid JWT access token is required.